

Assignment-19.1

HT.NO:2303A51731

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

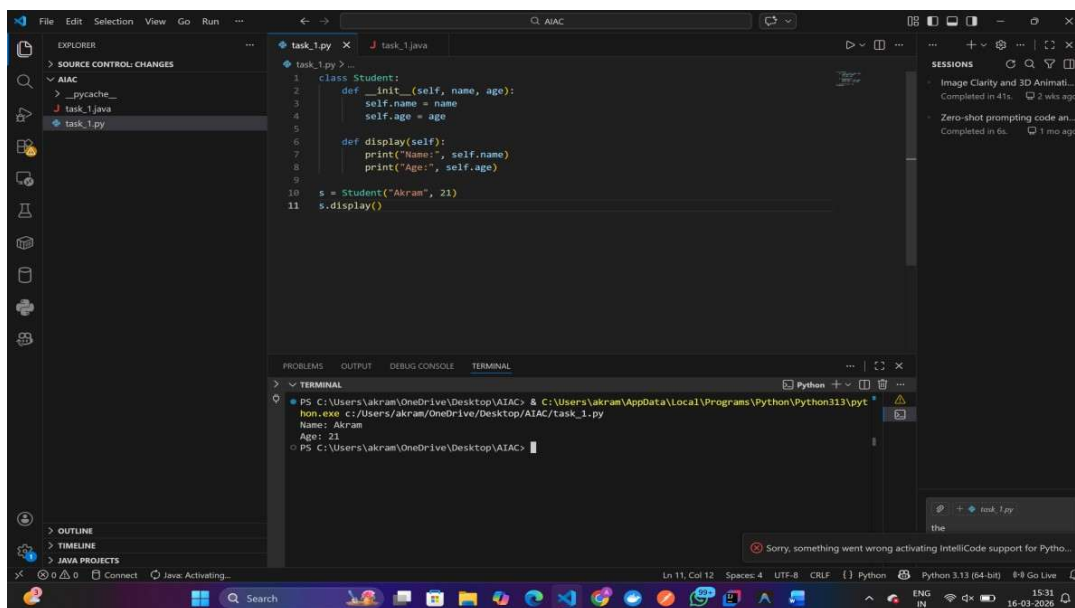
Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

CODE:

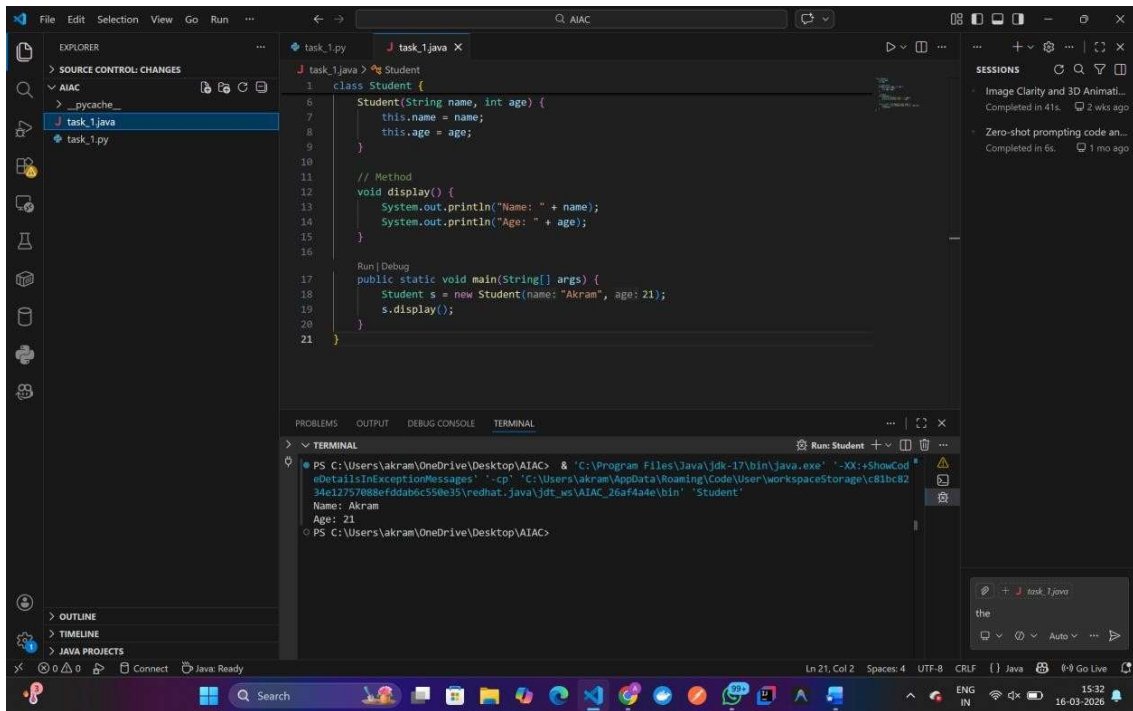


The screenshot shows a code editor with a Python file named `task_1.py`. The code defines a `Student` class with an `__init__` method that takes `name` and `age` as arguments, and a `display` method that prints the name and age. An instance `s` of the `Student` class is created with the name "Akram" and age 21, and the `display` method is called.

```
1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def display(self):
7         print("Name:", self.name)
8         print("Age:", self.age)
9
10 s = Student("Akram", 21)
11 s.display()
```

The terminal output shows the execution of the Python script, which prints the name and age of the student:

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```



Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

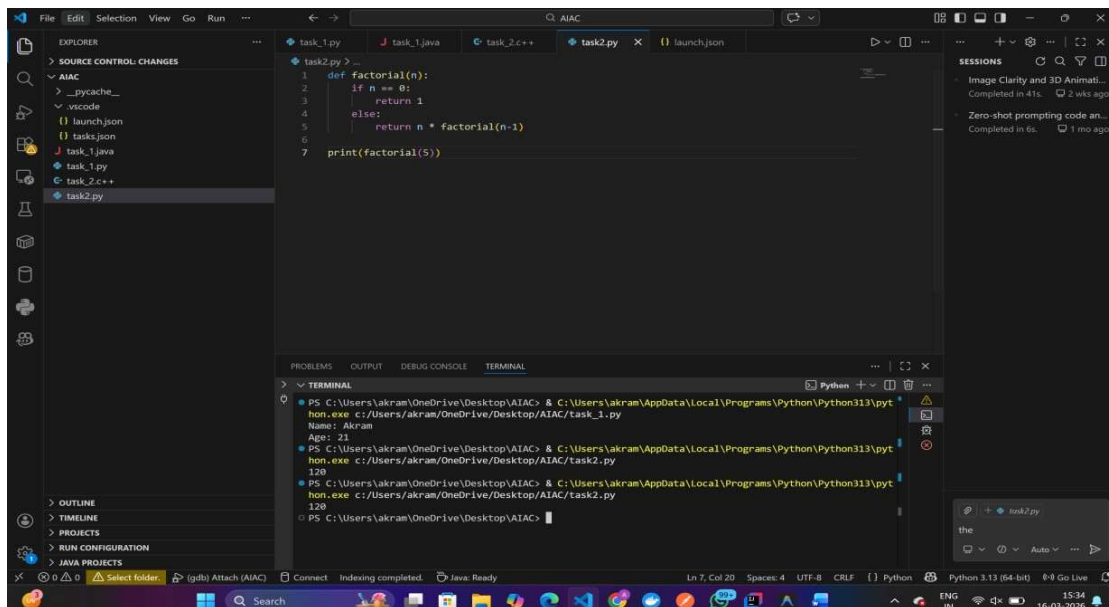
Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

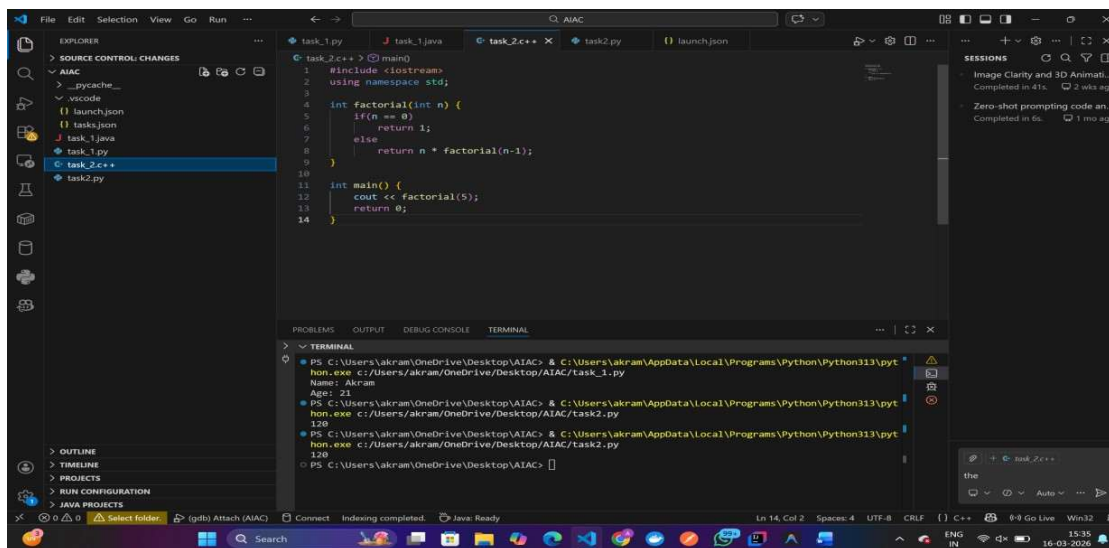
- A correct Python implementation equivalent to the original C++ factorial function.

CODE:



```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
print(factorial(5))
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```



```
#include <iostream>
using namespace std;

int factorial(int n) {
    if(n == 0)
        return 1;
    else
        return n * factorial(n-1);
}

int main() {
    cout << factorial(5);
    return 0;
}
```

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_1.py
Name: Akram
Age: 21
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:/Users/akram/OneDrive/Desktop/AIAC/task_2.py
120
PS C:\Users\akram\OneDrive\Desktop\AIAC>
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

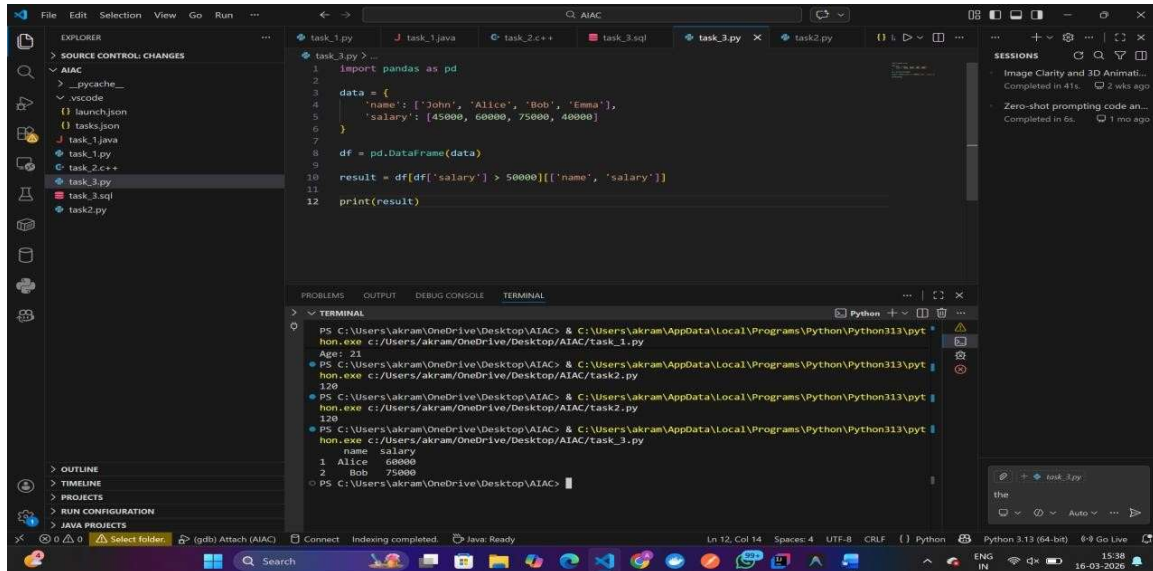
Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

CODE:

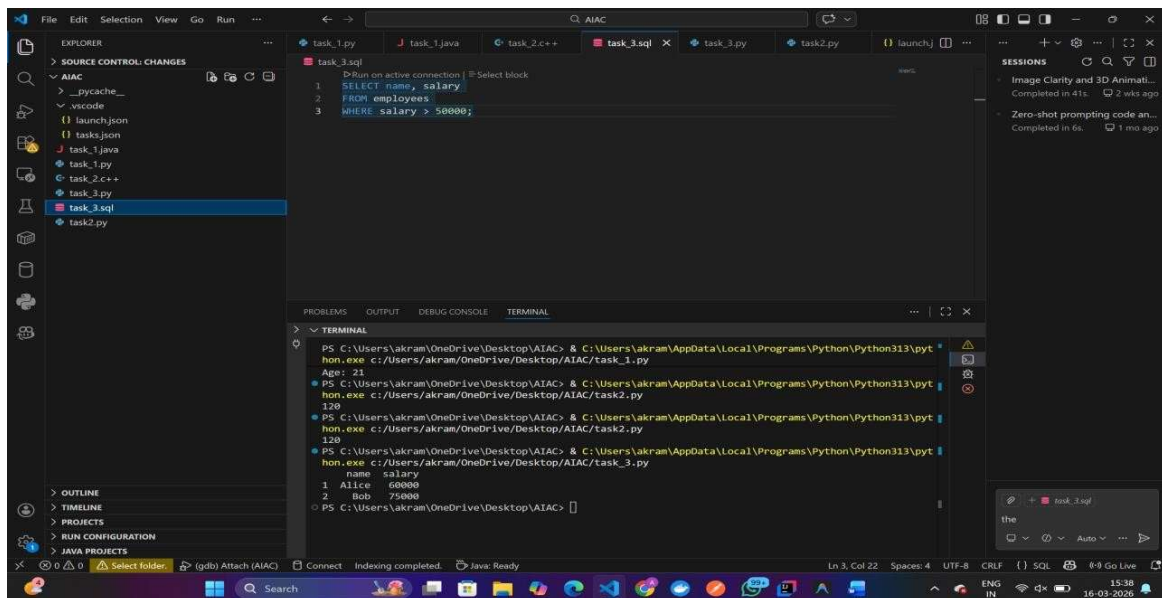


The screenshot shows the Visual Studio Code editor with a Python file named `task_3.py` open. The code defines a dictionary `data` with names and salaries, creates a `DataFrame`, and filters for salaries greater than 50,000. The terminal shows the execution of the script, displaying the filtered data as a table.

```
1 import pandas as pd
2
3
4 data = {
5     'name': ['John', 'Alice', 'Bob', 'Emma'],
6     'salary': [45000, 60000, 75000, 40000]
7 }
8 df = pd.DataFrame(data)
9
10 result = df[df['salary'] > 50000][['name', 'salary']]
11
12 print(result)
```

Terminal Output:

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_3.py
Age: 21
1  Alice  60000
2  Bob    75000
```



The screenshot shows the Visual Studio Code editor with a SQL file named `task_3.sql` open. The code contains a SQL query to select names and salaries from an `employees` table where the salary is greater than 50,000. The terminal shows the execution of the script, displaying the filtered data as a table.

```
1 SELECT name, salary
2 FROM employees
3 WHERE salary > 50000;
```

Terminal Output:

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python313\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_3.py
Age: 21
1  Alice  60000
2  Bob    75000
```

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages CODE:

The screenshot shows a Visual Studio Code editor window with a Python file named `task_4.py` open. The code in the editor is as follows:

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = "https://www.onlinegdb.com/"
5
6 response = requests.get(url)
7 soup = BeautifulSoup(response.text, "html.parser")
8
9 for link in soup.find_all('a'):
10     print(link.get("href"))
```

The terminal at the bottom shows the command to run the script and its output:

```
PS C:\Users\akram\OneDrive\Desktop\AIAC> & C:\Users\akram\AppData\Local\Programs\Python\Python311\python.exe c:\Users\akram\OneDrive\Desktop\AIAC\task_4.py
#editor_1
#
#tab-stdin
#tab-stdout
#tab-stderr
#tab-debug-console
/upgrade-premium
https://www.onlinegdb.com/blog/brief-guide-on-how-to-use-onlinegdb-debugger/
#debug_window_call_stack
#debug_window_local_variables
#debug_window_registers
#debug_window_display_expression
#debug_window_breakpoints
#
#
```

The status bar at the bottom indicates the file is encoded in UTF-8, has 4 spaces, and is using the CRLF line ending. The Python version is 3.11 (64-bit).

